

CONTENTS

- 01 Running vs. Not Running
- 02 The full state machine
- 03 Equal-priority round robin
- 04 The tick interrupt's role
- 05 Unequal priorities
- 06 Blocking instead of spinning
- 07 Mixing time-slicing and preemption
- 08 Creating and deleting tasks
- 09 A busier system, end to end

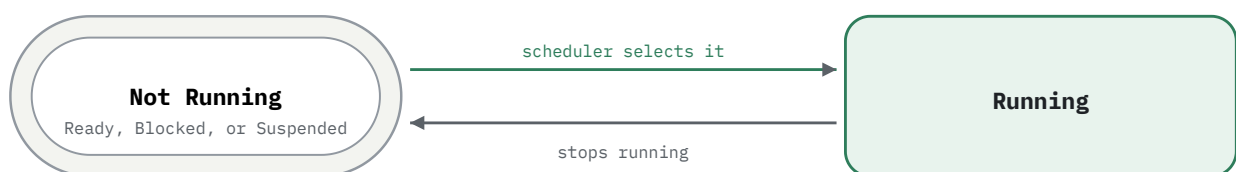
Original notes · on the subject of FreeRTOS task management

Task Management in FreeRTOS - a deeper pass

Nine execution patterns, each built from scratch to explain one specific idea - how the scheduler actually decides, tick by tick, which task gets the core. Not sourced from, or a redrawing of, any published book's artwork.

1 Running vs. Not Running

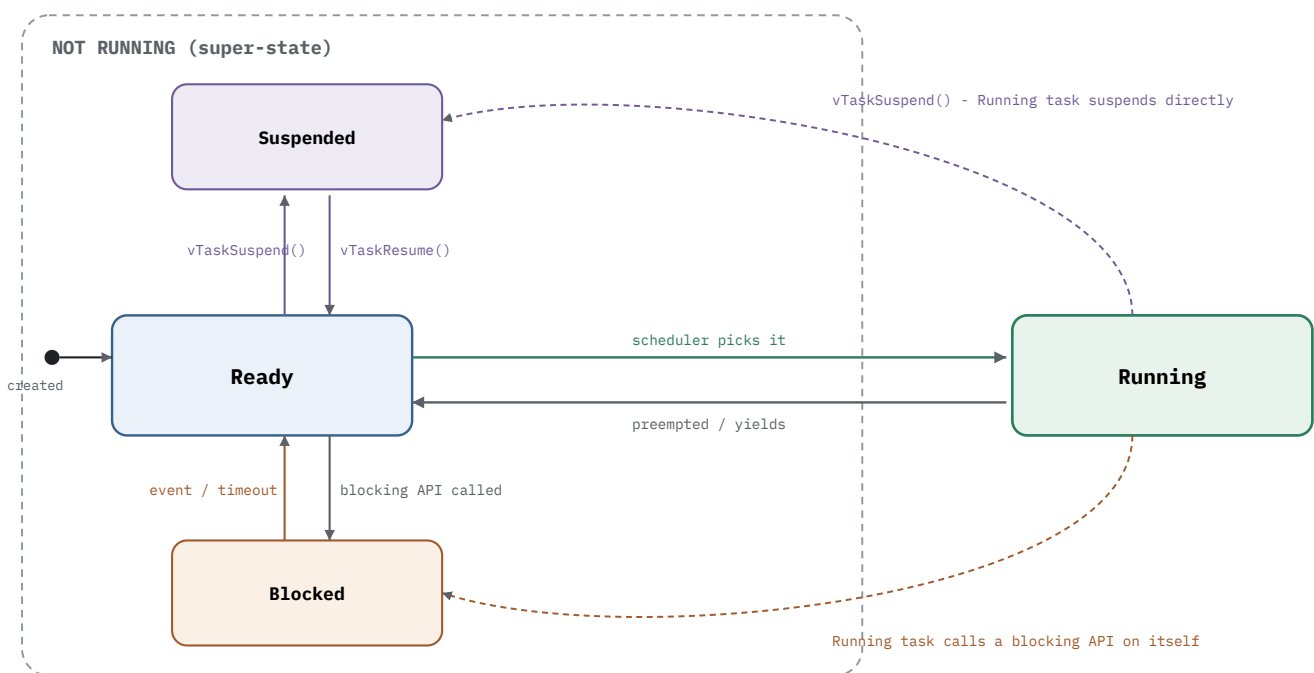
Before getting into all four states, the simplest possible view: at any instant, a task is either the one thing actually executing, or it isn't. Every other state - Ready, Blocked, Suspended - is just a different flavor of "not currently it."



Only one task in the entire system can be Running at a time (per core).
Everything else, no matter how close to running it is, is Not Running.

2 The full state machine

"Not Running" is really three distinct situations grouped under one umbrella. Ready means eligible right now; Blocked and Suspended are both forms of "not even eligible," for different reasons - a timeout that hasn't expired yet, versus removed from consideration entirely by another task's explicit request.

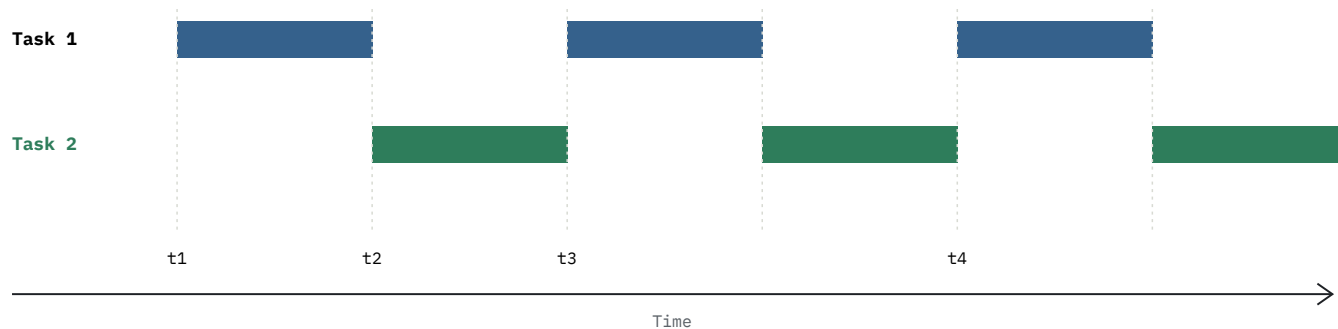


Every arrow is a specific API call or a specific event - nothing moves a task between states except one of these six triggers.

Notice what's symmetric and what isn't. Ready and Running trade places constantly, in both directions, many times a second. Suspended only ever exits back to Ready - never straight to Running - because `vTaskResume()` makes a task *eligible* again, it doesn't hand it the CPU on the spot.

3 Equal-priority round robin

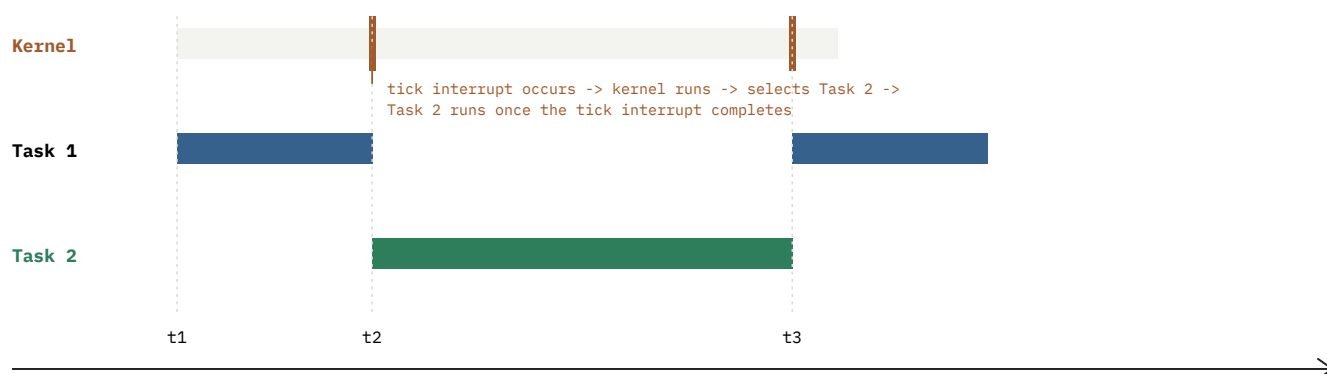
The simplest real execution pattern: two tasks, same priority, each doing nothing to block or yield. The scheduler just alternates them, one tick period each.



At t1, Task 1 enters Running and executes until t2. At t2 Task 2 enters Running and executes until t3, at which point Task 1 re-enters Running - and so on, indefinitely.

4 The tick interrupt's role

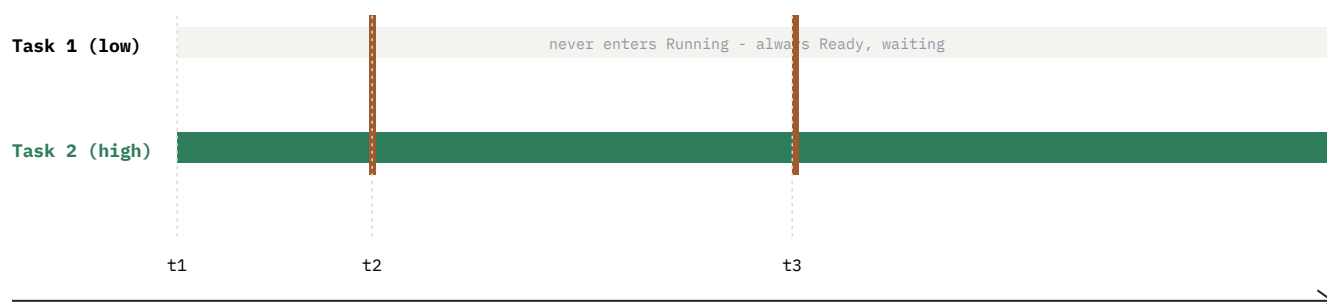
The switch in the diagram above isn't instantaneous or free - it's the kernel itself, briefly running inside a tick interrupt, deciding who goes next.



The kernel's own execution is real but brief - two narrow amber slivers, not a visible block of time like either task gets.

5 Unequal priorities

Change one thing - give Task 2 a higher priority than Task 1 - and the pattern above stops being an alternation at all. The tick interrupt still runs every time, but it keeps choosing the same task.

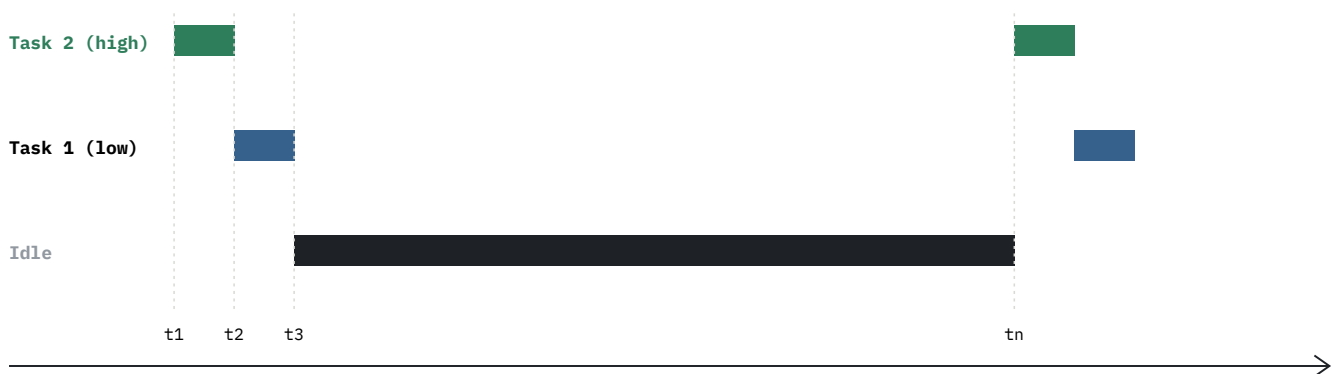


The tick interrupt at t_2 and t_3 still runs the scheduling decision - it just keeps arriving at the same answer, because nothing about Task 2's eligibility has changed.

This is a real hazard, not just an illustration. A higher-priority task that never blocks will starve everything below it, forever. It's the reason FreeRTOS tasks are almost always written to spend most of their time Blocked - waiting on a delay, a queue, or an event - rather than looping without ever giving anything up.

6 Blocking instead of spinning

The fix for the starvation pattern above: have each task voluntarily give up the CPU with `vTaskDelay()` once it's done its work for this pass, instead of looping forever. When every task is Blocked at once, a third task appears in the picture that's always there but easy to forget - the idle task.



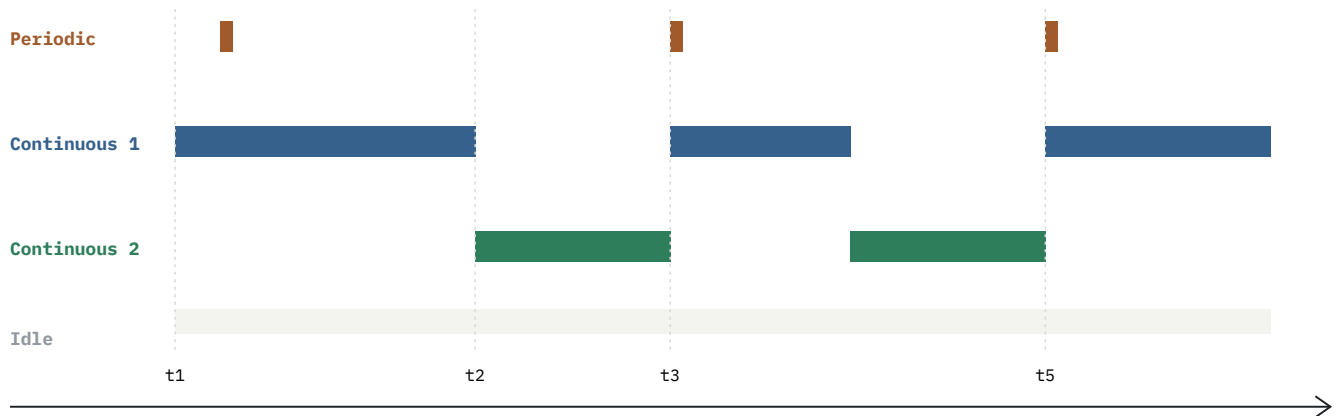
Task 2 (higher priority) runs first, prints, then blocks. Task 1 gets its turn, prints, then also blocks. With both tasks now Blocked, the idle task runs for the entire remaining gap, until Task 2's delay expires at t_n and the whole sequence repeats.

This is the same shape as `vApplicationIdleHook()` putting the core to sleep instead of spinning - covered in [the SysTick / PendSV notes](#) - except here the point is what makes the gap exist in the first place: two tasks that gave the CPU back voluntarily, rather than one that never had to.

7 Mixing time-slicing and preemption

Real systems combine both patterns at once. Here, two low-priority "continuous" tasks round-robin between themselves exactly like the two-task example earlier - but a

higher-priority "periodic" task cuts in every so often, runs briefly, and hands control straight back.

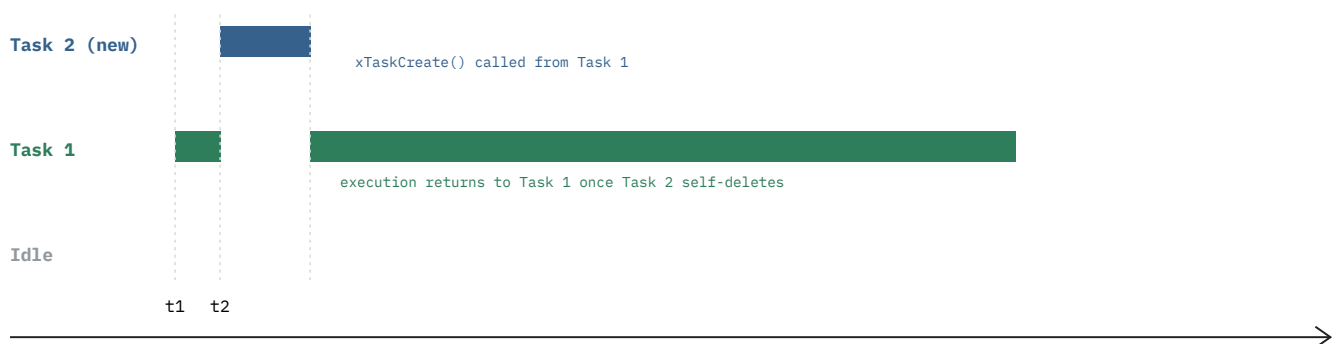


Continuous 1 and Continuous 2 share the CPU tick-by-tick between them, same priority, same as before. Periodic is higher priority than both - each time its block period expires, it preempts whichever continuous task is running, prints once, and immediately blocks again until next time.

Where idle fits. The idle task never appears as a real slice here, because Continuous 1 and Continuous 2 between them are *always* Ready - there's never a moment with nothing eligible to run below Periodic's priority.

8 Creating and deleting tasks

Tasks aren't fixed at compile time - one task can create another with `xTaskCreate()` at any point, and a task can delete itself (or be deleted by another) with `vTaskDelete()`.



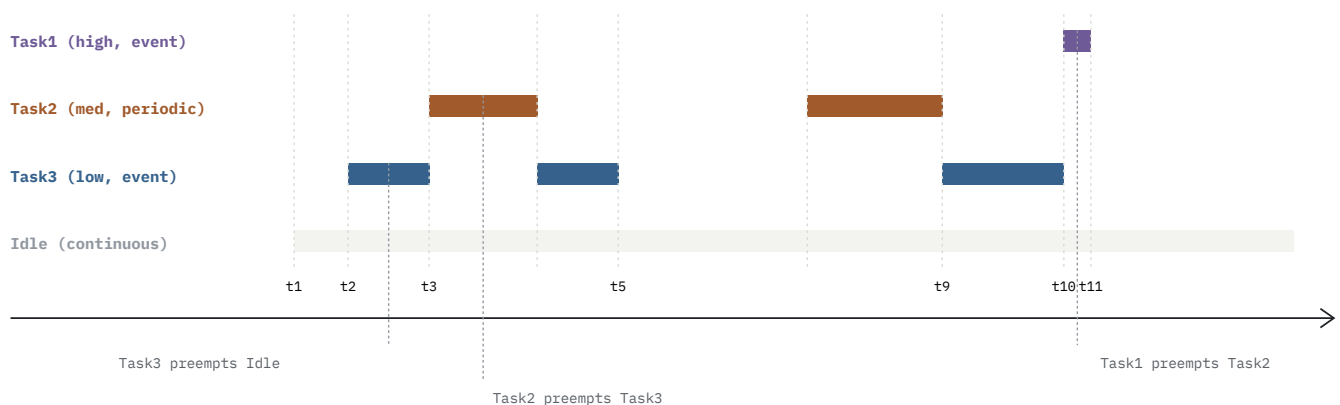
Task 1 runs, creates Task 2 at a higher priority. Task 2 immediately preempts it, does nothing but delete itself, and control returns to Task 1 - which then blocks, and the sequence repeats from the top.

The task that calls `vTaskDelete(NULL)` on itself never actually frees its own memory - it can't, while still standing on its own stack. That cleanup is deferred to the idle task, the next time it runs. A system where nothing ever calls `vTaskDelete()` from a task that's

never given the idle task a chance to run will leak memory quietly, one deleted task at a time.

9 A busier system, end to end

Pulling everything together: four priority levels, a mix of event-driven and periodic tasks, several preemption points in a row. Nothing new conceptually - just more of the same rules, layered.



Every colored block preempts whatever was running beneath it in priority - Task3 cuts into idle, Task2 cuts into Task3, Task1 cuts into Task2. Between t3 and t9, event-driven Task3 handles two separate bursts, each still bounded above by Task2's periodic priority.

The general rule, stated once. Every one of the nine diagrams above is really the same three sentences applied over and over: the highest-priority Ready task runs. A task becomes Ready by an event, a timeout, or being created. A running task stops being Running by blocking, being preempted, being suspended, or being deleted. Everything else is bookkeeping.

Original notes on FreeRTOS task management · execution patterns are illustrative examples, not reproductions of any specific published figure